

EXPERIMENT 7

Retrieval-Augmented Generation (RAG) System with Vector Database and LLM

AIM:

To integrate a Vector Database with an LLM to build a Retrieval-Augmented Generation (RAG) system that answers questions based on external PDF documents using LangChain, FAISS, and HuggingFace models.

PROCEDURE:

1. Install required libraries: langchain, langchain-community, langchain-huggingface, langchain-text-splitters, faiss-cpu, sentence-transformers, pypdf.
2. Import PyPDFLoader, RecursiveCharacterTextSplitter, HuggingFaceEmbeddings, FAISS, AutoTokenizer, AutoModelForSeq2SeqLM, and torch.
3. Upload a PDF document (e.g., research paper or LinkedIn profile).
4. Load the PDF using PyPDFLoader and extract all pages.
5. Split the document into chunks using RecursiveCharacterTextSplitter (chunk_size=500, overlap=50).
6. Generate embeddings for all chunks using HuggingFaceEmbeddings('all-MiniLM-L6-v2').
7. Build a FAISS vector store from the document chunks and embeddings.
8. Create a retriever from the FAISS store (k=3 nearest chunks).
9. Load a sequence-to-sequence LLM (e.g., google/flan-t5-base) using AutoTokenizer and AutoModelForSeq2SeqLM.
10. Define rag_answer(question): retrieve top-3 relevant chunks, concatenate as context, build prompt, generate answer using LLM.
11. Test with 3 domain-specific questions from the uploaded PDF.

CODE:

```
!pip install langchain langchain-community langchain-huggingface \
        langchain-text-splitters faiss-cpu sentence-transformers pypdf --quiet
```

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch
```

```
from google.colab import files
uploaded = files.upload()
pdf_file = list(uploaded.keys())[0]
```

```
loader = PyPDFLoader(pdf_file)
documents = loader.load()
print(f'Loaded {len(documents)} pages from PDF')
```

```
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(documents)
print(f'Split into {len(chunks)} chunks')
```

```
embeddings = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={'k': 3})
```

```
model_name = 'google/flan-t5-base'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

```
def rag_answer(question):
    docs = retriever.get_relevant_documents(question)
    context = ' '.join([d.page_content for d in docs])
    prompt = f'Context: {context}\nQuestion: {question}\nAnswer:'
    inputs = tokenizer(prompt, return_tensors='pt',
                       truncation=True, max_length=512)
    with torch.no_grad():
        outputs = model.generate(**inputs, max_new_tokens=150)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

questions = [
    'What is the main topic of this document?',
    'What are the key findings mentioned?',
    'Who are the authors or contributors?'
]

for q in questions:
    print(f'Q: {q}')
    print(f'A: {rag_answer(q)}')
    print('-' * 60)
```

OUTPUT:

? Question: What is this document about?

? Answer: Modeling semantic relationships

? Question: Give a summary of the roadmap

? Answer: The LLM pipeline consists of: Text Tokens Embeddings Transformer Logits Softmax Probabilities Each stage is explained below.

RESULT:

The RAG system was successfully implemented by integrating FAISS vector database with a HuggingFace sequence-to-sequence LLM. The system loaded and chunked an external PDF document, stored embeddings in a FAISS index, retrieved the top-3 most relevant chunks for each question, and generated grounded answers using the flan-t5-base model. The system effectively demonstrated how retrieval-augmented generation reduces hallucinations by anchoring responses to document-specific context.